

UNIVERSIDAD DE COSTA RICA

ESCUELA DE INGENIERÍA ELÉCTRICA

IE0117: PROGRAMACIÓN BAJO PLATAFORMAS ABIERTAS

Reporte

Laboratorio # 3

Prof.Carolina Trejos Quiros

Estudiante: Jafet Guillermo Cruz Salazar - C02520

3 de mayo de 2025

Índice

1. Introducción	2
2. Implementación	3
3. Resultados	10
4. Conclusiones y Recomendaciones	15
Referencias	16

1. Introducción

En el presente informe se abordan aspectos fundamentales de programación en lenguaje C relacionados con matrices y arreglos. A través de la resolución de tres ejercicios se fortalecieron las habilidades en el uso de estructuras de datos, la validación de entradas por parte del usuario y la identificación y corrección de errores.

En el contexto de la programación C, es importante manejar con precaución las operaciones relacionadas con matrices y arreglos, dado que errores como accesos fuera de los límites del arreglo pueden conducir a fallos críticos o vulnerabilidades de seguridad.

Como guía se tomo de referencia el manual de programación para C proporcionado por la IBM[1]

El código fuente de los scripts desarrollados se encuentra disponible en el siguiente repositorio; en este también se encuentran instrucciones para la ejecución de cada script: https://github.com/MavrosAilouros/Lab3_Data_structs_Arrays

2. Implementación

Ejercicio 1: Suma de diagonales de matrices cuadradas

Objetivo: Calcular la suma de las diagonales principal y secundaria de una matriz cuadrada con parámetros establecidos por el usuario.

Desarrollo: El ejercicio se realizó tomando en cuenta la minimización de introducción de datos erróneos con un código enfocado en la revisión y confirmación de dichos datos; por su parte el programa indica de manera robusta el método de introducción de datos al usuario, asegurando la entrada correcta de estos.

El script recibe y verifica los siguientes argumentos:

- El tamaño de la matriz cuadrada.
- Los valores “N” de cada una de las filas.
- verifica que los valores introducidos sean números enteros.
- verifica que el tamaño de la matriz se encuentre en el limite indicado por el código del programa.

En primera instancia, el código contiene cuatro encabezados de bibliotecas C estándar, cada una con su función específica, permitiendo el manejo eficiente de `input/output`, `memoria`, `cadenas` y límites numéricos, dichos encabezados son:

Listing 1: Encabezados de biblioteca

```
#include <stdio.h>
#include <stdlib.h>      /* strtol, EXIT_FAILURE */
#include <string.h>      /* fgets, strtok, strcspn */
#include <limits.h>      /* INT_MIN, INT_MAX */
```

La función de cada una de estas bibliotecas viene indicada en la documentación proporcionada por la IBM (por su siglas en inglés, International Business Machines).

Posteriormente se maneja la petición realizada al usuario para el valor “N” de la matriz y verificación de dicho número. Esto es posible mediante el comando `fgets` que se encarga de leer y almacenar la línea ingresada por el usuario. El texto ingresado se convierte en un número (`long`) usando `strtol` y se verifica que este sea correcto.

Listing 2: Lectura y validación de N

```
for (;;) {
    printf("Introduce el
    .....tamano_de_la_matriz
    .....cuadrada_(1- %d): ", MAX_N);

    if (!fgets(line, sizeof line, stdin))      /* EOF / error */
        return EXIT_FAILURE;

    line[strcspn(line, "\n")] = '\0';          /* trim \n */

    char *end;
```

```

    long val = strtol(line, &end, 10);

    if (*end != '\0')
        fprintf(stderr, "Entrada_invalida.
        .....Debes_escribir_un_numero_entero.\n");
    else if (val <= 0 || val > MAX_N)
        fprintf(stderr, "El_tamano_debe_estar
        .....entre_1_y_%d.\n", MAX_N);
    else { N = (int)val; break; }           /* valid N */
}

```

Una vez se valida “N” se declara la matriz “N x N” y se procede con la solicitud de escritura por filas en la matriz en cada línea, asegurando que las filas contengan exactamente “N” enteros y que cada valor sea un entero válido dentro del rango permitido y si esta fila no cumple lo solicitado, lo vuelve a pedir; por cada valor válido ingresado el programa avanza a la siguiente fila.

Se utilizan comandos los comandos **row** para almacenar la línea completa ingresada por el usuario, **token** para dividir la línea en números y **end** para validar si el **token** es un número entero.

Listing 3: Lectura y verificación de filas de Matriz

```

char    row[ROWBUF];
char    *token, *end;
long    v;

for (int i = 0; i < N; /* i++ only when the row is correct */) {

    printf("Fila_%d_(escribe_%d
    .....enteros_separados_por_espacios):\n",
           i + 1, N);

    if (!fgets(row, sizeof row, stdin)) {
        fprintf(stderr, "Error_de_lectura.\n");
        return EXIT_FAILURE;
    }

    int j = 0;
    token = strtok(row, "\t\r\n");

    while (token && j < N) {
        v = strtol(token, &end, 10);

        if (*end != '\0' || v < INT_MIN || v > INT_MAX) {
            j = -1;           /* Marks Invalid row */
            break;
        }
        matrix[i][j++] = (int)v;
        token = strtok(NULL, "\t\r\n");
    }
}

```

```

        if (j == N && token == NULL)           /* Correct row */
            ++i;                               /* Moves on to the next row */
        else
            fprintf(stderr,
                "Entrada_invalida._Debes_escribir
EXACTAMENTE_%d_numeros_enteros_validos_en
una_sola_linea.\n", N);
    }

```

Para la ultima sección del programa, se calcula y muestra la suma de las diagonales de la matriz cuadrada; se declaran variables para acumular datos y con el fin de evitar el desbordamiento debido a valores muy grandes se utiliza `long`. Finalmente se recorre cada una de las filas para acceder a cada uno de los elementos de las diagonales y mostrar sus resultados.

Listing 4: Suma y muestra de resultados

```

/* ----- Sum the diagonals ----- */
long maindiag = 0, secdiag = 0;

for (int i = 0; i < N; ++i) {
    maindiag += matrix[i][i];
    secdiag  += matrix[i][N - 1 - i];
}

/* ----- Display results ----- */
printf("\nSuma_de_la_diagonal_principal:_%d\n", maindiag);
printf("Suma_de_la_diagonal_secundaria:_%d\n", secdiag);
printf("Suma_de_ambas_diagonales:_%d\n", maindiag + secdiag);

return 0;
}

```

El algoritmo implementado permitió calcular la suma de las diagonales principal y secundaria de una matriz cuadrada de tamaños variables. Gracias al uso de estructuras de control simples y una validación robusta de entrada, el programa garantiza resultados correctos incluso en casos donde las dimensiones son grandes o la matriz contiene valores negativos. Además, la implementación fue verificada con múltiples casos de prueba, cumpliendo con todos los requerimientos funcionales establecidos en el enunciado del laboratorio.

Ejercicio 2: Corrección en error de lógica

Objetivo: El segundo ejercicio consistió en analizar y solucionar el problema presente en el código encargado de mostrar el numero máximo en un arreglo de números proporcionado; donde, al inicializar el programa sin cambios, este seleccionaba el numero mas pequeño de dicho arreglo.

Desarrollo:

El código original comparaba los elementos del arreglo con el valor actualmente almacenado en la variable máximo utilizando el operador <. Esta lógica causaba que se guardaran los valores menores en lugar de los mayores, lo que resultaba en la obtención del valor mínimo en lugar del máximo. Para corregir este problema, se modificó la condición lógica del if, reemplazando el operador < por >

Listing 5: Algoritmo corregido

```
int encontrarMaximo(int arr[], int n) {
    int maximo = arr[0];

    for (int i = 1; i < n; i++) {
        if (arr[i] > maximo) {           /* Corrected mistake: use '>' */
            maximo = arr[i];
        }
    }

    return maximo;
}
```

Esta corrección resulta en la inicialización de la variable máximo con el primer elemento del arreglo, y luego recorre el resto del arreglo desde la posición inicial hasta la final, comparando el valor actual con máximo y reemplazándolo si el valor actual es mayor. Su funcionamiento fue validado utilizando el arreglo proporcionado, y el programa devolvió correctamente “40” como el valor máximo.

Esta solución, aunque sencilla, es un gran ejemplo de como un pequeño error en la lógica puede cambiar completamente el comportamiento de un programa.

Ejercicio 3: Analisis de consecutividad en patrones binarios

Objetivo: Para este ejercicio se implemento un algoritmo con el objetivo de encontrar la racha mas larga de “1’s” consecutivos que aparezcan en las diagonales de una matriz binaria cuadrada.

Desarrollo: La solución se trabajó en dos etapas principales:

- El análisis de todas las diagonales posibles de la matriz, en ambas direcciones: descendente (de arriba a abajo) y ascendente (de abajo hacia arriba).
- La extensión del programa para admitir la generación aleatoria de datos binarios.

Para el análisis de las diagonales descendentes, se implementaron dos bucles diferenciados:

- El primero recorre todas las diagonales que comienzan en la fila 0, avanzando desde la columna 0 hasta la columna $N - 1$.
- El segundo recorre las diagonales que comienzan en la columna 0, desde la fila 1 hasta la fila $N - 1$, con el objetivo de no repetir la diagonal principal.

De forma análoga, se implementaron otros dos bucles para el análisis de las diagonales ascendentes:

- El primero recorre las diagonales que comienzan en la primera columna, desde la fila $N - 1$ hasta la fila 0.
- El segundo recorre las diagonales que comienzan en la última fila, desde la columna 1 hasta la columna $N - 1$, evitando así repetir la diagonal principal ascendente.

Para el primer bloque se implementa la lógica que recorre las diagonales de la matriz tanto en sentido descendente como ascendente.

La validación cubre todas las posibles diagonales únicas de una matriz cuadrada, garantizando que se analicen todas las permutaciones relevantes. Esto permite identificar correctamente la diagonal con la secuencia continua de “1’s” más larga.

Dentro de cada diagonal, el programa utiliza un contador para identificar la cantidad de “1’s” consecutivos. Si se encuentra un 0, el contador se reinicia. Si la longitud actual supera el valor previamente almacenado como el más largo (`maxLen`), dicho valor se actualiza.

Listing 6: Algoritmo de analisis

```
int findLargestLine(int m[][SIZE])
{
    int maxLen = 0;

    /* Descending diagonals that begin in column 0 */
    for (int col = 0; col < SIZE; ++col) {
        int len = 0;
        for (int r = 0, c = col; r < SIZE && c < SIZE; ++r, ++c) {
            if (m[r][c] == 1) {
                ++len;
                if (len > maxLen) maxLen = len;
            } else {
                len = 0;
            }
        }
    }
}
```



```

    }
  }
}

/* Descending diagonals that start from rows \.../
for (int startRow = 1; startRow < SIZE; ++startRow) {
    int len = 0;
    for (int r = startRow, c = 0; r < SIZE && c < SIZE; ++r, ++c) {
        if (m[r][c] == 1) {
            ++len;
            if (len > maxLen) maxLen = len;
        } else {
            len = 0;
        }
    }
}

/* Ascending diagonals that begin in the first column */
for (int row = SIZE - 1; row >= 0; --row) {
    int len = 0;
    for (int r = row, c = 0; r >= 0 && c < SIZE; --r, ++c) {
        if (m[r][c] == 1) {
            ++len;
            if (len > maxLen) maxLen = len;
        } else {
            len = 0;
        }
    }
}

/* Ascending diagonals that begin from the last row ... */
for (int col = 1; col < SIZE; ++col) {
    int len = 0;
    for (int r = SIZE - 1, c = col; r >= 0 && c < SIZE; --r, ++c) {
        if (m[r][c] == 1) {
            ++len;
            if (len > maxLen) maxLen = len;
        } else {
            len = 0;
        }
    }
}

/* Returns the length of the longest diagonal sequence of 1s */
return maxLen;

```

Para mostrar en consola el contenido de la matriz generada, se implementó la función auxiliar `printMatrix()`. Esta función recibe como parámetro una matriz cuadrada de tamaño fijo y se encarga de imprimirla en un formato ordenado y legible para el usuario.

Listing 7: Impresión de la matriz binaria en la terminal

```
void printMatrix(int m[][SIZE])
{
    puts("La matriz utilizada corresponde a:");
    for (int i = 0; i < SIZE; ++i) {
        for (int j = 0; j < SIZE; ++j)
            printf("%d", m[i][j]);
        putchar('\n');
    }
    putchar('\n');
}
```

Finalmente, para extender el programa, se utilizó la función `rand` de la biblioteca estándar de C, junto con `srand(time(NULL))`, las cuales permiten la generación de números pseudoaleatorios. En cada llamada a `rand()`, se devuelve un número entero comprendido entre 0 y `RAND_MAX`. La función `srand()` establece una semilla inicial para el generador, y al combinarla con `time(NULL)`, se garantiza una semilla única basada en el reloj del sistema. Esto permite la generación de matrices diferentes cada vez que se ejecuta el programa. Finalmente, el operador módulo `% 2` se utilizó para asegurar que los números generados fueran únicamente 0 o 1, manteniendo así la matriz binaria, tal como lo exige el enunciado.

Listing 8: Generacion de matriz binaria aleatoria con `rand()` y semilla temporal

```
#if RAND_FILL

    srand((unsigned)time(NULL));
    for (int i = 0; i < SIZE; ++i)
        for (int j = 0; j < SIZE; ++j)
            matrix[i][j] = rand() % 2;
#endif

    printMatrix(matrix);

    int largestLine = findLargestLine(matrix);

    printf("El tamaño de la secuencia en diagonal de...",
           largestLine);

    return 0;
```

El ejercicio permitió aplicar técnicas de análisis para detectar patrones en estructuras. La ampliación del código para considerar las secuencias repetitivas garantiza un análisis completo y preciso. Además la introducción de generación aleatoria de matrices refuerza la validación de dicho algoritmo.

3. Resultados

Ejercicio 1: Suma de diagonales de matrices cuadradas

A continuación se presentan los resultados obtenidos después de la compilación y ejecución del programa `Ejercicio1.c` en diferentes escenarios:

- Caso 1: Matriz 3x3 básica.
- Caso 2: Matriz 4x4 con ceros.
- Caso 3: Matriz 5x5 con todos sus elementos en 1.

```
[mavros_ailouros@fedora LaboratorioIII]$ ./ej1
Introduce el tamaño de la matriz cuadrada (1-10): 3

Guía de entrada: escribe cada fila en una sola línea.

1. Incluye exactamente los números enteros y de la matriz
2. Separados por espacios.
3. Presiona <Enter> para introducir los numeros.

Fila 1 (escribe 3 enteros separados por espacios):
1 2 3
Fila 2 (escribe 3 enteros separados por espacios):
4 5 6
Fila 3 (escribe 3 enteros separados por espacios):
7 8 9

Suma de la diagonal principal: 15
Suma de la diagonal secundaria: 15
Suma de ambas diagonales: 30
[mavros_ailouros@fedora LaboratorioIII]$
```

Figura 1: Ejecución del progrmama con matriz 3x3.

```
[mavros_ailouros@fedora LaboratorioIII]$ ./ej1
Introduce el tamaño de la matriz cuadrada (1-10): 4

Guía de entrada: escribe cada fila en una sola línea.

1. Incluye exactamente los números enteros y de la matriz.
2. Separados por espacios.
3. Presiona <Enter> para introducir los numeros.

Fila 1 (escribe 4 enteros separados por espacios):
0 0 0 1
Fila 2 (escribe 4 enteros separados por espacios):
0 0 1 0
Fila 3 (escribe 4 enteros separados por espacios):
0 1 0 0
Fila 4 (escribe 4 enteros separados por espacios):
1 0 0 0

Suma de la diagonal principal: 0
Suma de la diagonal secundaria: 4
Suma de ambas diagonales: 4
[mavros_ailouros@fedora LaboratorioIII]$
```

Figura 2: Ejecución del programa con matriz 4x4.

```
[mavros_ailouros@fedora LaboratorioIII]$ ./ej1
Introduce el tamaño de la matriz cuadrada (1-10): 5

Guía de entrada: escribe cada fila en una sola línea.

1. Incluye exactamente los números enteros y de la matriz.
2. Separados por espacios.
3. Presiona <Enter> para introducir los numeros.

Fila 1 (escribe 5 enteros separados por espacios):
1 1 1 1 1
Fila 2 (escribe 5 enteros separados por espacios):
1 1 1 1 1
Fila 3 (escribe 5 enteros separados por espacios):
1 1 1 1 1
Fila 4 (escribe 5 enteros separados por espacios):
1 1 1 1 1
Fila 5 (escribe 5 enteros separados por espacios):
1 1 1 1 1

Suma de la diagonal principal: 5
Suma de la diagonal secundaria: 5
Suma de ambas diagonales: 10
[mavros_ailouros@fedora LaboratorioIII]$
```

Figura 3: Ejecución del programa con matriz 5x5.

Ejercicio 2: Corrección en error de lógica

El siguiente conjunto de capturas muestra la ejecución del programa `Ejercicio2.c` compilado y su correcto funcionamiento.

```
[mavros_ailouros@fedora LaboratorioIII]$ ./ej2
El número más grande es: 40
[mavros_ailouros@fedora LaboratorioIII]$
```

Figura 4: Correcto funcionamiento del programa.

Ejercicio 3: Análisis de secuencia en patrones binarios

A continuación se presentan los resultados obtenidos después de compilar y ejecutar el programa `Ejercicio3.c` a partir de los siguientes casos.

- Caso 1: Prueba con Matriz de ejemplo proporcionada.
- Caso 2: Prueba cambiando el numero del operador `SIZE` por 5
- Caso 3: Prueba cambiando el numero de `SIZE` por 4

```
[mavros_ailouros@fedora LaboratorioIII]$ gcc -Wall Ejercicio3.c -o ej3
[mavros_ailouros@fedora LaboratorioIII]$ ./ej3
La matriz utilizada corresponde a:
1 0 1 0 1 1
1 0 1 1 1 0
0 1 1 1 1 1
1 1 1 1 1 1
0 0 1 1 1 0
0 0 0 1 0 0

El tamaño de la secuencia en diagonal de 1's más grande es: 4
[mavros_ailouros@fedora LaboratorioIII]$
```

Figura 5: Resultado de la prueba con la matriz ejemplo.

```
[mavros_ailouros@fedora LaboratorioIII]$ gcc -Wall Ejercicio3.c -o ej3
[mavros_ailouros@fedora LaboratorioIII]$ ./ej3
La matriz utilizada corresponde a:
0 1 0 0
0 0 1 1
0 1 0 1
1 1 0 0

El tamaño de la secuencia en diagonal de 1's más grande es: 3
[mavros_ailouros@fedora LaboratorioIII]$
```

Figura 7: Resultado de prueba con matriz SIZE = 4

```
[mavros_ailouros@fedora LaboratorioIII]$ gcc -Wall Ejercicio3.c -o ej3
[mavros_ailouros@fedora LaboratorioIII]$ ./ej3
La matriz utilizada corresponde a:
1 1 0 0 0
1 0 1 1 0
0 1 1 1 1
1 0 0 1 1
0 1 1 1 1

El tamaño de la secuencia en diagonal de 1's más grande es: 4
[mavros_ailouros@fedora LaboratorioIII]$
```

Figura 6: Resultado de prueba con matriz SIZE= 5.

4. Conclusiones y Recomendaciones

Conclusiones

El desarrollo de este laboratorio permitió fortalecer habilidades clave en programación estructurada con lenguaje C, aplicando conceptos esenciales como el uso de arreglos, validación robusta de entradas, y recorrido eficiente de estructuras matriciales. A lo largo de los tres ejercicios, se reforzaron conocimientos sobre:

- Declaración y manipulación de matrices cuadradas, así como el recorrido de diagonales en distintas orientaciones para detectar patrones.
- Implementación de lógica condicional para localizar valores máximos, con especial atención a errores comunes como comparaciones invertidas.
- Generación de datos pseudoaleatorios utilizando `rand()` y `srand(time(NULL))`, permitiendo probar el algoritmo con diferentes configuraciones de entrada.
- Separación del código en funciones auxiliares para mejorar la organización, legibilidad y mantenimiento del programa.

Además, se logró identificar y corregir errores lógicos y de formato en tiempo de desarrollo, destacando la importancia del proceso de depuración, pruebas iterativas y verificación manual de los resultados obtenidos.

Recomendaciones

- Validar exhaustivamente las entradas del usuario para garantizar la integridad de los datos antes de operar sobre ellos.
- Mantener una nomenclatura clara y consistente para las variables, especialmente en bucles anidados y recorridos de matrices.
- Comentar las secciones clave del código, particularmente aquellas que implementan lógica de búsqueda o manipulación de estructuras.
- Utilizar valores constantes definidos mediante `#define` para facilitar la modificación del tamaño de las matrices sin necesidad de alterar múltiples líneas de código.
- Incluir funciones auxiliares como `printMatrix()` para facilitar el proceso de depuración y verificación de resultados.
- Cuando se trabaje con matrices aleatorias, realizar múltiples ejecuciones para validar la robustez del algoritmo ante distintas configuraciones posibles.

Referencias

1. IBM Corporation, “C and c++ language support,” 2023. Accedido: 1 de mayo de 2025.